Experiment 1: Two Water Jug Problem

Aim / Objective: Understand the meaning of state space, to automate solving of the "**two water jug problem**" by taking user inputs and to learn and apply fundamental concepts of logical flow in Python.

Theory

The two water jug problem can be defined as:

1) It has two water jugs with different water capacities.
2) We can pour from one jug to another, empty and fill both the jugs at any time.
3) The end goal is to get a "required capacity" in either of the jugs.

Constraints:

1) The two jugs cannot overflow.
2) We can only fill amounts equal to capacities in the jug, but we can empty any and pour any amount.

Here, a tuple (J1, J2), where J1 and J2 are capacities of the jugs represent a state. Our end goal in this particular problem is to get 4 litres of water in either jugs. This is the goal state. The user has to reach this goal state by using the given functions. To solve the problem, we must apply a sequence of valid operations while ensuring that all constraints are satisfied at every step. Each action performed changes the current state into a new state, and the collection of all reachable states forms the state space of the problem. By carefully choosing moves such as filling, emptying, or transferring water between the jugs, we can explore different states. The process will continue until the desired goal state is achieved successfully without violating any rules.

Algorithm / Methodology

1) Start
   Read cap1, cap2, target
   Set j1 ← 0, j2 ← 0

2) Display menu
   1 Fill1
   2 Fill2
   3 Empty1
   4 Empty2
   5 Pour1→2
   6 Pour2→1
   0 Stop

3) Loop
   Print (j1, j2)

If (j1 = target OR j2 = target) → Go to Step 5
Read move

4) Switch(move)
   1 → j1 ← cap1
   2 → j2 ← cap2
   3 → j1 ← 0
   4 → j2 ← 0
   5 → t ← min(j1, cap2 − j2)
   j1 ← j1 − t
   j2 ← j2 + t
   6 → t ← min(j2, cap1 − j1)
   j2 ← j2 − t
   j1 ← j1 + t
   0 → Go to Step 5

5) Stop

Methodology: The water jug implemented as a state-space user-driven solution tuple system each represented a program modular where (j1, j2) approach a functional initial using separate state-space functions is (0, 0). For operator transitions fill, empty, and are defined pour. Deterministic functions transformation state using performances are transitions. Continuously loop menu-driven accepts a user-selected actions interface. Python's handled match-case control flow is constructed using switch-case. A set data structure visited is to store states ensuring tracking explored configurations. Defined condition termination reaching is a goal state where either jug contains target volume.

Dataset Description

The user choices itself make the datasets of the problem. The choices are what determines how the goal state is reached. Every state is a result of user chosen operation.
Attributes: These include the operations to fill, empty, pour, the capacities of two water jugs and whether the final state is reached.
Size: The size of operation depends on the user selected sequence of operations.

Implementation

```
# Function definitions implementing all valid water jug operators
(fill, empty, pour) to generate state transitions in the problem space

def fill1(capOf1):
    return capOf1
```

```python
def fill2(capOf2):
    return capOf2

def empty1():
    return 0

def empty2():
    return 0

def pour12(jug1, jug2, cap2):
    transfer = min(jug1, cap2 - jug2)
    return jug1 - transfer, jug2 + transfer

def pour21(jug1, jug2, cap1):
    transfer = min(jug2, cap1 - jug1)
    return jug1 + transfer, jug2 - transfer

# Main function manages user interaction, menu-driven control flow
using match-case, state updates, goal-state validation, and tracking of
all visited configurations

def main():
    cap1 = int(input("Enter capacity of jug 1: "))
    cap2 = int(input("Enter capacity of jug 2: "))
    target = int(input("Enter target amount: "))

    j1 = 0
    j2 = 0

    # Set used to store and track all visited states
    visited = set()
    visited.add((j1, j2))

    print("1 Fill Jug1")
    print("2 Fill Jug2")
    print("3 Empty Jug1")
    print("4 Empty Jug2")
    print("5 Pour Jug1 to Jug2")
    print("6 Pour Jug2 to Jug1")
    print("0 Stop")

    while True:
        print("Current State:", (j1, j2))
```

```
        if j1 == target or j2 == target:
            print("Target reached")
            break

        move = int(input("Enter move: "))

        match move:
            case 1:
                j1 = fill1(cap1)
            case 2:
                j2 = fill2(cap2)
            case 3:
                j1 = empty1()
            case 4:
                j2 = empty2()
            case 5:
                j1, j2 = pour12(j1, j2, cap2)
            case 6:
                j1, j2 = pour21(j1, j2, cap1)
            case 0:
                break
            case _:
                print("Invalid move")
                continue

        visited.add((j1, j2))

    print("All states visited:")
    for s in visited:
        print(s)
```

Output / Result

```
main()

Enter capacity of jug 1: 3
Enter capacity of jug 2: 5
Enter target amount: 4
1 Fill Jug1
2 Fill Jug2
3 Empty Jug1
4 Empty Jug2
5 Pour Jug1 to Jug2
6 Pour Jug2 to Jug1
0 Stop
Current State: (0, 0)
Enter move: 1
Current State: (3, 0)
Enter move: 2
Current State: (3, 5)
Enter move: 4
Current State: (3, 0)
Enter move: 5
Current State: (0, 3)
Enter move: 1
Current State: (3, 3)
Enter move: 6
Current State: (3, 3)
Enter move: 5
Current State: (1, 5)
Enter move: 4
Current State: (1, 0)
Enter move: 5
Current State: (0, 1)
Enter move: 1
Current State: (3, 1)
Enter move: 5
Current State: (0, 4)
Target reached
All states visited:
(0, 1)
(0, 4)
(0, 0)
(1, 5)
(3, 1)
(0, 3)
(3, 0)
(3, 3)
(1, 0)
(3, 5)
```

Result Analysis

In the above result, we get to the goal state (0,4) in 11 steps. The states that come up are printed. The user determines which sequence of operations to take. The program stops when we reach the goal state.

Conclusion

This program demonstrates the two water jug problem solution in a state space approach. The code uses a choice based interface, providing the user with the power to determine the sequence of the solution. This experiment helps learn about AI topics such as problem solving by defining a goal state and using state space search.

Experiment 1: Missionaries and Cannibals

Aim / Objective: Understand the meaning of state space, to automate solving of the "Missionaries and Cannibals problem" by taking user inputs and to learn and apply fundamental concepts of logical flow in Python.

Theory

The missionaries and cannibals problem is an Artificial Intelligence problem used to demonstrate state space representation and constraint-based problem solving. It consists of three missionaries and three cannibals standing on a bank of a river. A boat is available that can carry at most two people at a time. The goal is to transfer all missionaries and cannibals to the opposite bank of the river safely..

The main constraint of the problem is that at no time should the number of cannibals exceed the number of missionaries on either side of the river, if missionaries are present. If this condition is violated, the missionaries would be in danger. The boat must always carry at least one person and can carry either one or two individuals per trip.

A state in this problem is represented using a tuple (M, C, B), where M represents the number of missionaries on the left bank, C represents the number of cannibals on the left bank, and B represents the position of the boat (0 for left bank and 1 for right bank). The initial state is (3, 3, 0) and the goal state is (0, 0, 1).

The user must reach the goal state by applying valid operations. Each move transforms the current state into a new state, and the collection of all valid reachable states forms the state space. By carefully choosing valid movements of missionaries and cannibals, the goal state can be achieved without violating constraints.

Algorithm / Methodology

1) Start
   Set M ← 3, C ← 3, B ← 0
2) Display menu
   1 Move 1 Missionary
   2 Move 2 Missionaries
   3 Move 1 Cannibal
   4 Move 2 Cannibals
   5 Move 1 Missionary and 1 Cannibal
   0 Stop

3) Loop
   Print (M, C, B)
   If (M = 0 AND C = 0 AND B = 1) → Go to Stop
   Read move
4) Switch(move)
   1 → Apply (1M) move
   2 → Apply (2M) move
   3 → Apply (1C) move
   4 → Apply (2C) move
   5 → Apply (1M1C) move
   Check validity of new state
   If valid → update state
   Else → reject move
5) Stop

Methodology: The missionaries and cannibals problem implemented as a state-space user-driven solution tuple system each represented a program modular where (M, C, B) approach a functional initial using separate state-space functions is (3, 3, 0). For operator transitions movement functions are defined for valid combinations of missionaries and cannibals. Deterministic functions transformation state using performances are transitions. Continuously loop menu-driven accepts a user-selected actions interface. Python's handled match-case control flow is constructed using switch-case. A set data structure visited is to store states ensuring tracking explored configurations. Defined condition termination reaching is a goal state where all missionaries and cannibals safely reach the opposite river bank.

Dataset Description

The user choices themselves make the dataset of the problem. The choices are what determine how the goal state is reached. Every state is a result of a user-chosen operation.

Attributes: These include the number of missionaries, number of cannibals, boat position, type of move selected, and whether the final state is reached.

Size: The size of the dataset depends on the user-selected sequence of valid movements.

Implementation

```
# Function definitions implementing valid state transitions for the
missionaries and cannibals problem
def move(m, c, boat, dm, dc):
    return m - dm, c - dc, 1 - boat

def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if (m > 0 and m < c):
        return False
    if (3 - m > 0 and 3 - m < 3 - c):
        return False
    return True
# Main function manages user interaction, menu-driven control flow
using match-case, state updates, goal validation, and tracking of
visited states
def main():
    m = 3
    c = 3
    boat = 0

    visited = set()
    visited.add((m, c, boat))

    print("1 Move 1 Missionary")
    print("2 Move 2 Missionaries")
    print("3 Move 1 Cannibal")
    print("4 Move 2 Cannibals")
    print("5 Move 1 Missionary and 1 Cannibal")
    print("0 Stop")

    while True:
        print("Current State:", (m, c, boat))

        if m == 0 and c == 0 and boat == 1:
            print("Goal reached")
            break

        move_choice = int(input("Enter move: "))

        match move_choice:
            case 1:
```

```
                nm, nc, nb = move(m, c, boat, 1 if boat == 0 else -1,
0)
            case 2:
                nm, nc, nb = move(m, c, boat, 2 if boat == 0 else -2,
0)
            case 3:
                nm, nc, nb = move(m, c, boat, 0, 1 if boat == 0 else
-1)
            case 4:
                nm, nc, nb = move(m, c, boat, 0, 2 if boat == 0 else
-2)
            case 5:
                nm, nc, nb = move(m, c, boat, 1 if boat == 0 else -1, 1
if boat == 0 else -1)
            case 0:
                break
            case _:
                print("Invalid move")
                continue

        if is_valid(nm, nc):
            m, c, boat = nm, nc, nb
            visited.add((m, c, boat))
        else:
            print("Invalid state")

    print("All states visited:")
    for s in visited:
        print(s)
```

Output / Result

```
main()

1 Move 1 Missionary
2 Move 2 Missionaries
3 Move 1 Cannibal
4 Move 2 Cannibals
5 Move 1 Missionary and 1 Cannibal
0 Stop
Current State: (3, 3, 0)
Enter move: 4
Current State: (3, 1, 1)
Enter move: 3
Current State: (3, 2, 0)
Enter move: 4
Current State: (3, 0, 1)
Enter move: 3
Current State: (3, 1, 0)
Enter move: 2
Current State: (1, 1, 1)
Enter move: 5
Current State: (2, 2, 0)
Enter move: 2
Current State: (0, 2, 1)
Enter move: 3
Current State: (0, 3, 0)
Enter move: 4
Current State: (0, 1, 1)
Enter move: 3
Current State: (0, 2, 0)
Enter move: 4
Current State: (0, 0, 1)
Goal reached
All states visited:
(0, 2, 1)
(0, 0, 1)
(0, 2, 0)
(3, 1, 0)
(0, 3, 0)
(0, 1, 1)
(3, 3, 0)
(3, 0, 1)
(1, 1, 1)
(2, 2, 0)
(3, 1, 1)
(3, 2, 0)
```

Result Analysis

In the above result, the program successfully reaches the goal state (0, 0, 1) through a valid sequence of user-selected moves. The states generated during execution are displayed and tracked. The user controls the movement strategy while the system ensures that invalid or unsafe states are rejected. The program terminates when the goal state is achieved.

Conclusion

This experiment demonstrates the missionaries and cannibals problem using a state space approach. The solution uses a menu-driven interface that allows the user to explore different valid state transitions. The implementation helps in understanding important Artificial Intelligence concepts such as state representation, constraints, goal testing, and systematic problem solving using logical flow in Python.